Linear Algebra for Data Science

Course project 2026: Brief, timeline, assessment and topic ideas

Rostyslav Hryniv

Ukrainian Catholic University
Data Science Master Programme

1st term
Autumn 2026



APPLIED SCIENCES FACULTY

Outline

1. The project
2. Timeline and assessment
3. Topic ideas

Outline

1 The project

2 Timeline and assessment

3 Topic ideas

What the project is

The task

Choose an applied problem you care about. Find the linear algebra inside it, learn the algorithms used to solve it, implement them, and test them honestly.

Why it carries 25 points

There is no final exam. The project is where you show the main skill of the course: given an applied problem, identify the linear-algebraic structure, choose the factorization, predict its cost and failure modes, and implement it.

Format

Teams of 2–3. Report, code and a recorded presentation, due mid-December; peer review in January.

What your team will do

1. State the problem and why it matters in practice.
2. Formulate it in linear algebra: what is the matrix, what is unknown, which factorization or eigenproblem is involved.
3. Survey the available algorithms and choose at most two.
4. Explain the linear algebra behind them in your own words.
5. Predict their cost and their failure modes: conditioning, stability, scale.
6. Implement them; compare against a library routine and a benchmark.
7. Report what worked, what did not, and why.

The one requirement: the LA test

The range of topics is deliberately broad. The only requirement is that the project rests essentially on linear-algebra concepts and algorithms. A topic passes if your report answers all three questions:

The LA test

1. **Object.** Which matrix, subspace, factorization or eigenproblem is at the centre of the problem?
2. **Algorithm.** Which linear-algebra algorithm solves it, and why does it work, at the level of this course?
3. **Your code.** Which part of it do you implement yourselves, rather than only call from a library?

The lecturer checks the LA test when approving your topic.

The LA test: two examples

Passes

"Recommending films by low-rank matrix completion: alternating least squares versus singular value thresholding." Object: a partially observed ratings matrix of low rank. Algorithms: two. Code: both.

Fails as stated

"Predicting house prices with gradient boosting." The data form a matrix, but no linear-algebra object is the subject of the project.

The same data, reframed to pass

"Ridge regression through the SVD: how the singular values of the design matrix control the regularization path, and the cost of recomputing it for many penalties."

What you submit

Report and code

- **Report:** 8–10 pages (PDF), excluding references, figures and appendices. Sections: problem; formulation; methods and cost; experiments; discussion.
- **Code:** repository or archive; the `README` says how to reproduce every figure and table.

Presentation and statements

- **Recorded presentation:** 10–12 minutes, every team member speaks, aimed at a classmate who has not read the report.
- **Contribution statement:** who did what, agreed by all members.
- **AI-use statement:** which tools, for what.

Implementation: implement the core yourselves

Your own implementation

Write the central algorithm from `numpy` building blocks: matrix products, triangular solves, norms. Python is the default; any language is fine.

A fair comparison

Compare it with the library routine that does the same job (`scipy.linalg`, `sklearn`, `torch`, ...) and, where one exists, with a published benchmark.

Honest failure is credited

A method that fails, with a clear explanation of *why* it fails, scores better than clean output with no understanding behind it.

Experiments: what to measure

Measure what the course taught you to measure

- **Accuracy:** residuals, errors against a known answer, or a standard benchmark metric.
- **Conditioning:** $\mathrm{cond}(A)$ of the matrices you actually factor, and its effect on accuracy.
- **Cost:** runtime against problem size, compared with the flop count you predicted.
- **Limits:** the size or condition number at which your method stops working.

Example

A least-squares project reports $\mathrm{cond}(A)$ for its design matrix, compares the errors of the normal equations, QR and `lstsq`, and plots runtime against m next to the predicted $\sim 2mn^2$.

Topics the course already covers

Allowed only if you go clearly beyond the course version: a new application, a much larger scale, or a different algorithm compared with the course one.

Lab 1	fitting via QR; DFT/DCT energy compaction; the perceptron
Lab 2	power iteration, Markov chains, PageRank; circulants and FFT
Lab 3	covariance and whitening; Cholesky sampling; kernel ridge
Lab 4	SVD compression and denoising; PCA and MDS; deblurring
Lab 5	CG and preconditioning; randomized SVD; spectral clustering

Example. “PageRank on a small graph” is Lab 2. “Personalized PageRank on the Wikipedia link graph: power iteration versus a Krylov solver” is a project.

A warning about LLM topics

Our advice: avoid topics about large language models

LLMs are full of matrices, but they make poor linear-algebra projects:

- The linear algebra is buried: most of the effort goes into pipelines, GPUs, prompts and hyperparameters.
- You cannot implement the core yourselves or run controlled experiments on a laptop, so question 3 of the LA test fails.
- Results depend on compute and tuning more than on the method, so cost and failure modes cannot be isolated.
- The literature is mostly fast-moving preprints, which are hard to check.

If you still want one

It needs the lecturer's explicit approval at registration, one clearly isolated linear-algebra object, and experiments that run on a laptop.

Outline

1 The project

2 Timeline and assessment

3 Topic ideas

Timeline

Mon 19 Oct	**Team and topic registration** on Moodle
Thu 22 Oct	Topic approved, or returned with a suggested reframing
Mon 2 Nov	**Interim report 1**: scoping memo
Mon 16 Nov	**Interim report 2**: working baseline
Mon 14 Dec	**Final submission**: report, code, recording, statements
11–24 Jan	**Peer review** in Moodle Workshop
by 29 Jan	Staff grades; short online Q&A where needed

All deadlines 23:59 EET time

Two things to plan for

- Each interim report gets written feedback within a week.
- Lab 5 is due at the end of November; the three weeks after it are the main project window.

Registration and interim report 1

Registration: one paragraph

Team members; the application domain and the method family; the problem; the data or benchmark; the linear-algebra object; one or two candidate algorithms; two or three references.

Interim report 1: scoping memo, 2 pages

- the problem, the aim and the precise task;
- the linear-algebra formulation;
- possible approaches, their pros and cons, and their predicted cost;
- the chosen method(s), the plan, and who does what;
- 3–5 references you have actually read.

Interim report 2

Working baseline: 2 pages plus a link to the code

- a first implementation running on small data;
- a first comparison with a library routine, with one figure;
- the outline of the final report;
- what is not working yet, and any change of plan.

Why this milestone

A project whose code first runs in December has no time left to learn from its experiments. Interim report 2 moves that moment to November.

Grading: 25 points

Pts	Component	How it is marked
2	Interim 1	2 complete, passes the LA test; 1 needs revision; 0 missing
3	Interim 2	3 baseline runs, with results; 2 partial; 1 plan only; 0 missing
12	Staff grade	report, code, presentation; rubric on the next slide
4	Peer grade	classmates' marks for your report and presentation, same rubric
4	Review quality	the marks and comments *you* give
25		

Assessment criteria

Staff and peers use the same five criteria, each scored 0–3.

Criterion	A score of 3 means	Weight
A. Formulation	LA object stated precisely; method choice follows from it	2
B. Mathematics	key linear algebra explained correctly, in your own words; central claims proved or sketched	3
C. Cost, failure modes	cost predicted and checked; conditioning and stability discussed; limits found	2
D. Implementation	own core code; fair comparison with library and benchmark; reproducible	3
E. Communication	clear report, honest figures, a presentation a classmate can follow	2

Staff grade $= \frac{1}{3}(2A + 3B + 2C + 3D + 2E)$, out of 12. Full descriptors in the appendix.

Peer review: how it works

- It runs in Moodle Workshop, the tool used in the workshop tutorials 2.2, 3.2 and 6.2.
- Each student, individually, reviews two reports, with their recordings, from other teams.
- For each criterion you give a score 0–3 and at least two sentences of justification, then one paragraph of advice.

Peer review: how it is marked

Your review quality: 4 points

- 2 from Workshop: how close your scores are to the other reviewers' scores;
- 2 from staff: are your comments specific, correct and useful to the authors?

Safeguards

- Staff moderate the peer grade and override clear outliers.
- A review not submitted scores 0 for that review.

Teams, individual grades, lateness

Teams

Registration is first come, first served. Two teams may share a topic area only with clearly different problems or methods.

Individual grades

All members receive the team's grade by default. Staff may adjust an individual grade when the contribution statement, the presentation or a Q&A shows a clearly unequal contribution.

Late submissions

Interim reports: the course late policy; feedback is still given. Final submission: a report that misses the Workshop submission phase cannot be peer-reviewed, so its peer grade is 0.

AI tools

Allowed

Help with code, language editing, and finding literature.

Required

- Declare the use in the AI-use statement.
- Every member can explain every equation and every line of the core code.
- References are real and have been read. An invented reference is treated as a serious breach.

Outline

1 The project

2 Timeline and assessment

3 Topic ideas

Choosing a topic: two ways in

Route 1: start from an application domain

Audio and speech; image processing; vision-assisted navigation; recommender systems; physical phenomena (vibrations, diffusion, fluid flow, climate). Which linear-algebra problems does it keep solving?

Route 2: start from a type of method

Wavelet and other fast transforms; a factorization (QR, Cholesky, SVD, NMF, tensor); randomized linear algebra; iterative and Krylov solvers; spectral methods on graphs. Where does the method do real work?

Either way

A topic is a domain plus a method, with data you can obtain. Your work or your thesis is a good source of either. Then apply the LA test.

Domain plus method: examples

Domain	Methods	Example topic
Audio	short-time Fourier transform, wavelets, NMF	separating instruments in a recording by NMF of its spectrogram
Image processing	wavelets, SVD, sparse solvers	image compression with wavelets versus the DCT
Vision-assisted navigation	homogeneous least squares, SVD, sparse least squares	visual odometry: camera motion from the essential matrix
Recommender systems	low-rank completion, randomized SVD	alternating least squares versus singular value thresholding
Physical phenomena	eigenproblems, sparse solvers, SVD	dynamic mode decomposition of fluid-flow or traffic data

The idea slides that follow are grouped by the five themes of the course, with the linear-algebra core of each in brackets. They are examples, not a menu.

Where to start reading

- G. Strang, *Linear Algebra and Learning from Data*, Wellesley–Cambridge Press, 2019.
- L. N. Trefethen, D. Bau, *Numerical Linear Algebra*, SIAM, 1997.
- G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins, 2013.
- S. L. Brunton, J. N. Kutz, *Data-Driven Science and Engineering*, 2nd ed., Cambridge, 2022.
- P.-G. Martinsson, J. A. Tropp, Randomized numerical linear algebra: foundations and algorithms, *Acta Numerica* 29, 2020.

Survey papers and the documentation of the library you compare against (`scipy`, `sklearn`, ...) are often the fastest way into a topic.

Ideas: fitting, estimation, inverse problems

- **Kalman filtering as recursive least squares**, and its more stable square-root form [QR updates, Givens rotations]
- **Positioning from noisy distances**: GPS, Wi-Fi or UWB indoor positioning [nonlinear least squares, Gauss–Newton]
- **Instrumental variables and two-stage least squares**; “controlling for” as projecting out [projections]
- **Total least squares** when the inputs are noisy too [SVD]
- **Tomography**: Kaczmarz’s method versus CGLS [row projections, Krylov methods]
- **Optimal experimental design and sensor placement** [$\log\det$, Cholesky, A- and D-optimality]
- **Ranking from pairwise results**: Massey and Colley ratings for sports [least squares on graphs]

Ideas: spectra, Markov chains, graphs

- **Ranking beyond Lab 2**: HITS, personalized PageRank on a large real graph [eigenvectors, Krylov solvers]
- **Community detection on a real network**: normalized cuts, modularity [Laplacian eigenvectors, Lanczos]
- **Graph signal processing**: the graph Fourier transform and denoising on graphs [Laplacian eigenbasis]
- **Manifold learning**: Laplacian eigenmaps, diffusion maps [sparse symmetric eigenproblems]
- **How fast does a Markov chain mix?** Card shuffling, MCMC convergence [spectral gap]
- **The QR algorithm for eigenvalues**: Hessenberg reduction, shifts, deflation [Householder, QR]
- **Optimal transport by the Sinkhorn algorithm** [matrix scaling, positive matrices]

Ideas: symmetric and positive definite matrices

- **Gaussian process regression** and how to scale it [Cholesky, $\log\det$, low-rank approximations]
- **Kernel methods for large data**: Nystrom approximation versus random Fourier features [low-rank PSD matrices]
- **Covariance estimation in high dimension**: shrinkage (Ledoit–Wolf), the nearest correlation matrix [PSD cone, eigenvalues]
- **Portfolio optimization and factor models of risk** [quadratic forms, Cholesky, low rank plus diagonal]
- **Gaussian graphical models**: what zeros in the precision matrix mean [inverse covariance, Schur complements]
- **Diffusion models**: the forward noising process in closed form [Gaussian vectors, covariance]
- **Second-order optimization**: Gauss–Newton and natural gradient [Hessians, Fisher information, CG]

Ideas: low-rank structure

- **Recommender systems**: alternating least squares versus singular value thresholding [low-rank completion]
- **Topic modelling**: NMF versus latent semantic analysis [NMF, truncated SVD]
- **Word embeddings from co-occurrence counts**: PPMI plus SVD versus word2vec [SVD]
- **Compressing a small image classifier** by low-rank approximation of its layers [SVD, Eckart–Young]
- **Robust PCA**: video background subtraction [low rank plus sparse]
- **Tensor decompositions**: CP, Tucker, HOSVD [multilinear SVD]
- **Hidden factors**: ICA, CCA, probabilistic PCA, factor analysis [generalized eigenproblems]
- **Structure from motion**: the Tomasi–Kanade factorization on real video [rank-3 SVD]

Ideas: scale, sparsity, randomization

- **Randomized least squares**: sketch-and-precondition (Blendenpik, LSRN) versus QR [random projections, QR]
- **Vector databases**: random projections and the Johnson–Lindenstrauss lemma, product quantization [random matrices, nearest neighbours]
- **Image editing and deblurring with preconditioned CG** [sparse SPD systems, Krylov methods]
- **SLAM and bundle adjustment**: why sparsity and the Schur complement make them feasible [sparse least squares]
- **Sparse direct solvers**: fill-in and reordering (minimum degree, nested dissection) [LU, Cholesky, graphs]
- **Mixed precision**: solving in half precision, refining in double [LU, conditioning, iterative refinement]
- **Matrix-free deep learning**: Hessian-vector products and Lanczos for the loss-surface spectrum [Krylov methods]

Ideas: signals, audio, images, vision

- **Wavelets and image compression** (JPEG 2000) [orthogonal transforms, fast algorithms]
- **Audio source separation**: NMF of spectrograms [NMF, short-time Fourier transform]
- **Compressed sensing**: recovering sparse signals with orthogonal matching pursuit [projections, coherence]
- **Visual odometry for navigation**: the eight-point algorithm and the essential matrix [SVD, homogeneous least squares]
- **Speaker or face verification** with subspace methods [PCA, LDA, generalized eigenproblems]
- **Edge detection and image filtering** as linear operators [convolution, Toeplitz and circulant matrices]
- **Frequency estimation in radio signals**: MUSIC and ESPRIT [subspaces, eigenvectors of covariance matrices]

Ideas: real-world physical phenomena

- **Dynamic mode decomposition** of fluid-flow, traffic or epidemic data [SVD, eigenvalues]
- **Vibration modes of a bridge or a building** [generalized symmetric eigenproblem]
- **Heat diffusion and the Poisson equation on a grid** [sparse SPD systems, CG, FFT]
- **Climate patterns**: empirical orthogonal functions, the PCA of space-time fields [SVD]
- **Weather data assimilation**: the ensemble Kalman filter [low-rank covariance, least squares]
- **Electrical networks and Kirchhoff's laws** [graph Laplacian, least squares]

Topics from past years

- Discrete wavelet transform
- Edge detection algorithms
- Matrix factorization for topic modelling
- Military object tracking
- PageRank for image search
- Propaganda classification
- Speaker verification techniques
- UAV detection through RF analysis
- SVD applications in bioinformatics
- Investigation of the SURF algorithm
- Applications of spectral graph theory
- Frequency transforms for image recognition and inpainting
- Estimation of signal parameters
- Face recognition with NMF

These titles name the application only; a 2026 title names the linear algebra too. For example: “Detecting drones from radio spectra: subspace methods (MUSIC, ESPRIT) for frequency estimation”.

Summary

The requirement

Any applied problem whose solution rests essentially on linear algebra: name the object, explain the algorithm, implement its core. Start from a domain or a method; avoid LLM topics.

The next step

Register your team and topic on Moodle by Mon 19 October.
Questions: the course Slack channel, or at the start of any class.

Rubric descriptors: A and B

	3	2	1
A. Problem and formulation	problem and its practical value clear; matrix, unknowns and factorization stated precisely; method choice justified by the structure	formulation correct, but the link from structure to method choice is left implicit	LA appears only in passing, or the formulation is vague or wrong
B. Mathematics of the method	key ideas explained correctly in the team's own words; central claims proved or convincingly sketched; notation consistent; sources cited	correct but closely follows the sources, or has gaps in the key step	errors in the key step, or a paraphrase of sources without understanding

Rubric descriptors: C and D

	3	2	1
C. Cost and failure modes	flop count and memory predicted and checked; conditioning and stability discussed for the actual matrices; limits of the method found experimentally	cost stated but not checked, or conditioning mentioned without evidence	cost or limitations mentioned only in general terms
D. Implementation and experiments	core algorithm implemented by the team; fair comparison with a library routine and a benchmark on accuracy and runtime; every figure reproducible from the `README`	implementation works, but the comparison is thin or not fully reproducible	library calls only, or results that cannot be reproduced

Rubric descriptors: E, and how scores combine

	3	2	1
E. Communication	report readable by a classmate; figures labelled and honest; presentation within time, every member speaks, the main result is clear	mostly clear, with lapses in structure, figures or timing	hard to follow; key results buried or missing

A score of 0 in any criterion means missing, or not recognisable as an attempt.

From scores to points

- Staff grade $= \frac{1}{3}(2A + 3B + 2C + 3D + 2E)$, out of 12.
- Peer grade: the same weighted score from each reviewer, averaged and rescaled to 4.